

[00:00:01]

The video begins by revisiting the topic of creating EC2 instances and AWS S3 buckets from previous tutorials. The focus shifts today to two crucial services in AWS: **Load Balancing** and **Auto Scaling Groups (ASG)**. The speaker outlines the plan to cover what load balancers are, their types, differences, and practical implementation using AWS.

[00:01:36]

Load Balancing is introduced as the method to distribute incoming workloads or network traffic efficiently across multiple servers or instances. An analogy is used involving paint and load balancing to illustrate distributing work. The discussion starts by explaining EC2 instances as virtual servers with specific hardware capabilities (RAM, CPU), configured via instance types like **t2.large**. These servers run software such as Apache (**httpd**) to host websites.

[00:03:24]

The key problem addressed: if one EC2 instance can handle 10,000 concurrent users, increasing users (e.g., 15,000 users) requires more instances. Hence, creating multiple EC2 instances (at least 2-3) helps handle the increased load.

To distribute requests across these instances, the service called a **Load Balancer** is introduced. The load balancer distributes incoming network traffic evenly to prevent any single instance from becoming a bottleneck.

[00:04:55]

Advantages of Load Balancer:

- **High availability:** Even if one instance crashes or is stopped, the load balancer routes requests to remaining healthy servers, ensuring continuous service availability.
- Load balancers perform **health checks** and route traffic only to healthy instances.
- Load balancers allow **horizontal scaling** by managing multiple servers seamlessly.

[00:05:49]

The speaker emphasizes the role of a load balancer to distribute incoming traffic, which can vary unpredictably (from thousands to tens of thousands of users). Load balancing algorithms decide how the traffic is routed to backend servers.

[00:06:43]

AWS provides **three types of load balancers**:

- **Classic Load Balancer (CLB):** Legacy, supports HTTP, HTTPS, and TCP protocols, but considered outdated.
- **Application Load Balancer (ALB):** Operates at the OSI Layer 7 (Application Layer), supporting HTTP and HTTPS with advanced features like path-based and host-based routing.
- **Network Load Balancer (NLB):** Operates at OSI Layer 4 (Transport Layer), handling TCP and UDP traffic with high performance and static IP support.

Among these, the focus is primarily on ALB and NLB as CLB is being deprecated.

[00:07:13]

Classic Load Balancer handles HTTP, HTTPS, and TCP but is deprecated because ALB and NLB provide more advanced features and better performance. ALB is mostly used for web applications (HTTP/HTTPS), while NLB is used for high-performance TCP and UDP connections.

[00:10:12]

The speaker explains the HTTP/HTTPS protocols in detail:

- **HTTP (Hypertext Transfer Protocol):** Transmits data in plain text, used mainly for browsing websites.
- **HTTPS:** HTTP over TLS/SSL, adding a security layer that encrypts data to protect sensitive information like login credentials, credit card details during transmission.

He explains the importance of **SSL certificates** issued by Certificate Authorities which provide encryption and secure connections shown by the padlock symbol in browsers.

[00:15:43]

The difference between **TCP** and **UDP** protocols is explained in relation to the OSI model Layer 4:

- **TCP (Transmission Control Protocol):** Provides reliable, ordered, and error-checked delivery of data with a three-way handshake mechanism. Mainly used for file transfers, emails, and applications where data integrity is critical.
- **UDP (User Datagram Protocol):** Faster, connectionless, with no delivery guarantee, suitable for streaming, live video, online gaming, where some packet loss is tolerable.

[00:23:03]

A brief comparison between ALB and NLB is given from an AWS perspective:

- Use **ALB** for HTTP/HTTPS based web applications.
- Choose **NLB** for TCP/UDP traffic where ultra-high performance and static IP addresses are needed.
- The **Classic Load Balancer** supports both protocols but is outdated and should be avoided.

[00:25:35]

The video proceeds with a **demonstration of creating two EC2 instances** (named Green and Red), each running Apache on port 80 with simple HTML content. Each server hosts a different simple webpage showing "Green" or "Red" page content to visualize request distribution.

[00:37:28]

A **Classic Load Balancer (CLB)** is created and associated with these two EC2 instances. The

demonstration shows that when refreshing the browser, requests are distributed alternately to the Green and Red servers, verifying **load balancing with the Round Robin algorithm**.

[00:43:55]

Round Robin Algorithm is explained as a basic traffic distribution technique where requests are sent cyclically: the first request to server 1, second to server 2, and so forth, cycling back after the last server.

Load Balancing Algorithm	Description
Round Robin	Distributes requests sequentially and repeatedly across all servers.
Weighted Round Robin	Assigns weights to servers; servers with higher weight receive more requests.
Least Connections	Routes requests to the server with the fewest active connections.
Least Response Time	Routes requests to the server with the lowest response time.
IP Hash	Routes requests based on client's IP address to ensure session persistence.

[00:48:38]

The **Weighted Round Robin** algorithm enables assigning different priorities or capacities to backend servers. For example, a powerful server might get 60% of the traffic, while lower-capacity servers handle less.

[00:50:02]

Least Connections algorithm routes traffic to servers with the fewest active connections, balancing load dynamically based on current usage.

[00:53:00]

Shifting to **Application Load Balancer (ALB)**, which operates on HTTP/HTTPS (Layer 7). ALB supports advanced features such as:

- Health checks
- Logging
- Fault tolerance
- Load balancing to multiple ports on the same instance
- **Path-based routing**: Route traffic based on URL path (e.g., [/green](#), [/red](#))
- **Host-based routing** (routing based on domain name)
- **Sticky sessions** (session persistence)

These advanced features are not available in Classic or Network Load Balancers.

[00:59:04]

The concept of **Sticky Sessions** (session persistence) is explained:

- Useful for applications like banking where a session timeout occurs after inactivity.
- Sticky sessions ensure that requests from the same client IP address are consistently routed to the same backend instance for a specified timeout (e.g., 300 seconds).
- This helps maintain session consistency during activities like login, preventing session timeout issues.

[01:02:13]

The speaker compares ALB and NLB:

- **ALB** supports HTTP/HTTPS, path-based routing, host-based routing, sticky sessions, SSL certificates.
- **NLB** works at Layer 4 (TCP/UDP), providing high performance load balancing but without SSL termination or path-based routing.
- **NLB** supports static IP (Elastic IP) attachment, unlike ALB or CLB.

[01:19:29]

Next topic: **Auto Scaling Group (ASG)**

- Automatically adds or removes EC2 instances based on demand.
- Example: One EC2 instance can handle 10,000 requests, but if load increases to 20,000, ASG automatically launches new instances to handle extra traffic (horizontal scaling).
- When load decreases, ASG terminates unnecessary instances to optimize cost.

[01:21:08]

Differentiates between:

- **Vertical Scaling:** Upgrading hardware capacity of a single instance (e.g., from 16GB RAM to 32GB RAM).
- **Horizontal Scaling:** Adding more instances to share load; favored in AWS, implemented via **Auto Scaling Groups**.

[01:22:03]

Demonstrates creating an Auto Scaling Group using:

- A **Launch Template** (defining AMI, instance type, security groups).
- Settings for minimum, desired, and maximum instance counts.
- Scaling policies to govern when to add or remove instances based on demand.

[01:28:50]

Explains the **launch template** concept:

- AWS automatically spins up new EC2 instances based on this template when scaling out.
- The template specifies instance type, AMI, security group, startup scripts (e.g., to install Apache and create index pages).
- Enables easy replication and management of standardized instances during scaling.

[01:30:21]

When an instance is terminated, ASG automatically launches another instance to maintain the desired capacity, ensuring **high availability** and **fault tolerance**.

[01:32:09]

Highlights the synergy of Load Balancing and Auto Scaling for AWS architectures:

- Load Balancer distributes traffic efficiently to multiple instances.
- Auto Scaling dynamically adjusts the number of running instances based on load, ensuring optimal performance and cost efficiency.
- Together, they build **resilient and scalable** cloud applications.

[01:36:11]

Concludes by encouraging consistent review of notes and understanding of terms like sticky sessions, path-based routing, load balancing algorithms, and Auto Scaling best practices due to their relevance in real projects and interview scenarios. The speaker emphasizes the importance of learning and revising to master AWS concepts.

Summary Table: AWS Load Balancer Types and Features

Load Balancer Type	OSI Layer	Protocols Supported	Key Features	Static IP Support	Status
Classic Load Balancer	4 & 7	HTTP, HTTPS, TCP	Basic Load balancing, Health checks, Legacy	No	Deprecated
Application Load Balancer (ALB)	7	HTTP, HTTPS	Path-based routing, Host-based routing, Sticky sessions, SSL termination	No	Recommended
Network Load Balancer (NLB)	4	TCP, UDP	High performance, Static IP, Handles millions of requests	Yes	Recommended

Key Concepts Highlighted

- **Load Balancer:** Distributes incoming traffic to instances to prevent overload and ensure high availability.
 - **Auto Scaling Group:** Manages the automatic scaling (add/remove) of EC2 instances based on traffic demand.
 - **Round Robin Algorithm:** Default traffic distribution method cycling through servers sequentially.
 - **Weighted Round Robin:** Allows prioritization by assigning weights to instances.
 - **Sticky Sessions:** Maintains session affinity by routing a client to the same server for a set period.
 - **Path-Based Routing:** Routes traffic based on URL path prefixes to different target groups.
 - **Host-Based Routing:** Routes traffic based on hostname/domain to different target groups.
 - **HTTP/HTTPS Protocols:** HTTPS adds encryption and security to HTTP, protecting sensitive data.
 - **TCP vs UDP:** TCP guarantees reliable transmission; UDP is used for latency-sensitive applications like streaming and gaming.
 - **Launch Template:** Standardizes instance creation settings to enable reliable and consistent instance launch for Auto Scaling.
 - **Vertical vs Horizontal Scaling:** Increasing capacity of single server vs adding more servers to distribute load.
-

Interview-Relevant Insights

- Differentiate **ALB vs NLB** based on OSI layer, protocols, and supported features.
 - Explain **load balancing algorithms**: round robin, weighted round robin, least connections.
 - Define **sticky sessions** and why they are important.
 - Clarify the working mechanism of **TCP's three-way handshake**.
 - Demonstrate understanding of **Auto Scaling concepts**: launch templates, scaling policies, maintaining high availability.
 - Know the advantages of **horizontal scaling using Auto Scaling Groups** over vertical scaling.
 - Discuss practical AWS setup: creating EC2 instances, configuring load balancers, attaching target groups, and implementing path routing.
-

Professional Conclusion

This comprehensive tutorial offers a foundational yet detailed exploration of AWS load balancing and auto scaling mechanisms. By combining conceptual explanations, protocol distinctions, architectural patterns, and demo-based implementation steps, it equips learners with the

knowledge and skills necessary to architect scalable, reliable cloud applications on AWS. Understanding these components is not only essential for cloud infrastructure but is also critical for technical interviews focused on cloud computing and DevOps. The instructor encourages thorough note-taking and continual revision to build expertise.